

Lunyard

Backyard-Ultra Live-Dashboard

FSST-Projekt – Projektdokumentation

1. Projektidee & -konzept

Lunyard ist ein Live-Dashboard für ein Backyard-Ultra-Rennen.

Bei einem Backyard Ultra startet jede volle Stunde eine neue Runde. Wer die feste Strecke nicht rechtzeitig schafft, scheidet aus – der Letzte gewinnt.

Strava Beacon zeigt zwar Live-Position und Distanz jedes Läufers, rechnet aber nicht aus, wer seine Runde geschafft hat. Genau das macht unsere App:

- Sie zeigt mehrere Strava-Beacon-Livekarten gleichzeitig (z. B. am Beamer).
- Sie liest automatisch Name, Distanz und Zeit aus.
- Sie berechnet Runden, Pace und Ausscheidungen selbst.
- Unten läuft ein Info-Streifen für alle Zuschauer (Countdown, Status, Uhr).

2. Projektplanung (Feature-Liste)

Priorisierung: Muss (wichtig), Soll (geplant), Kann (Extra).

Feature	Priorität
Mehrere Livekarten anzeigen (1–8 Felder)	Muss
Backyard-Konfiguration (Startzeit, Rundenzeit, Strecke)	Muss
Runden über Timer steuern + Countdown	Muss
Ausgeschiedene Läufer erkennen und anzeigen	Muss
Daten automatisch aus Strava auslesen	Muss
Info-Streifen (Status, Fortschrittsbalken, Ortszeit)	Soll
Pace und Rundenzeit pro Person	Soll
Rundenzeit-Diagramm pro Läufer	Soll
Hell-/Dunkelmodus	Kann
Live-Vollbildmodus	Kann
Eigenes Icon / Start per Verknüpfung	Kann

3. Technologie Stack

- JavaScript – die gesamte Logik
- HTML & CSS – Oberfläche und Design
- Electron – macht aus Web-Technik eine Desktop-App
- Node.js – Laufzeitumgebung
- Strava Beacon – Datenquelle (Live-Standort)

- Tools: Visual Studio Code, Git, Claude, Cursor

4. Teamauflistung

- Fenkart, Benedikt
- Sukhrukov, Maksym

5. KI-Einsatz & Vorgehen

Wir haben das Projekt bewusst mit Unterstützung von Künstlicher Intelligenz umgesetzt, dabei aber großen Wert darauf gelegt, den Code selbst zu verstehen. Als Modell kam Claude Opus 4.8 zum Einsatz. Unser Vorgehen war iterativ: Statt alles auf einmal zu verlangen, haben wir die App Feature für Feature aufgebaut und für jeden Schritt einen möglichst klaren, präzisen Prompt geschrieben. Ein guter Prompt hat bei uns immer das Ziel, die gewünschten Funktionen und das erwartete Verhalten genau beschrieben – zum Beispiel „die Rundenzeit soll als echte Sekundenuhr ab dem Rundenstart laufen und einfrieren, sobald das Streckenziel erreicht ist“. Je genauer der Prompt, desto besser das Ergebnis. Nach jedem Schritt haben wir das Resultat geprüft: den Code gelesen, die App gestartet und getestet und bei Bedarf den Prompt nachgeschärft oder Korrekturen verlangt. So sind wir Schritt für Schritt vom Grundgerüst (mehrere Karten anzeigen) über die Strava-Auswertung bis zur kompletten Backyard-Logik gekommen. Die KI haben wir außerdem für Nebenaufgaben genutzt – etwa für das App-Icon, die Dokumentation und die Präsentation. Wichtig war uns dabei: Die KI liefert Vorschläge, die Entscheidungen und das Verständnis bleiben aber bei uns. Wir mussten den erzeugten Code nachvollziehen, einordnen und testen, damit am Ende ein Programm entsteht, das wir auch selbst erklären können.